

14.2 Data Structures for Graphs

In this section, we introduce four data structures for representing a graph. In each representation, we maintain a collection to store the vertices of a graph. However, the four representations differ greatly in the way they organize the edges.

- In an **edge list**, we maintain an unordered list of all edges. This minimally suffices, but there is no efficient way to locate a particular edge (u, v) , or the set of all edges incident to a vertex v .
- In an **adjacency list**, we additionally maintain, for each vertex, a separate list containing those edges that are incident to the vertex. This organization allows us to more efficiently find all edges incident to a given vertex.
- An **adjacency map** is similar to an adjacency list, but the secondary container of all edges incident to a vertex is organized as a map, rather than as a list, with the adjacent vertex serving as a key. This allows more efficient access to a specific edge (u, v) , for example, in $O(1)$ expected time with hashing.
- An **adjacency matrix** provides worst-case $O(1)$ access to a specific edge (u, v) by maintaining an $n \times n$ matrix, for a graph with n vertices. Each slot is dedicated to storing a reference to the edge (u, v) for a particular pair of vertices u and v ; if no such edge exists, the slot will store null.

A summary of the performance of these structures is given in Table 14.1.

Method	Edge List	Adj. List	Adj. Map	Adj. Matrix
numVertices()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
numEdges()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
vertices()	$O(n)$	$O(n)$	$O(n)$	$O(n)$
edges()	$O(m)$	$O(m)$	$O(m)$	$O(m)$
getEdge(u, v)	$O(m)$	$O(\min(d_u, d_v))$	$O(1)$ exp.	$O(1)$
outDegree(v) inDegree(v)	$O(m)$	$O(1)$	$O(1)$	$O(n)$
outgoingEdges(v) incomingEdges(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n)$
insertVertex(x)	$O(1)$	$O(1)$	$O(1)$	$O(n^2)$
removeVertex(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n^2)$
insertEdge(u, v, x)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$
removeEdge(e)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$

Table 14.1: A summary of the running times for the methods of the graph ADT, using the graph representations discussed in this section. We let n denote the number of vertices, m the number of edges, and d_v the degree of vertex v . Note that the adjacency matrix uses $O(n^2)$ space, while all other structures use $O(n + m)$ space.

14.2.1 Edge List Structure

The **edge list** structure is possibly the simplest, though not the most efficient, representation of a graph G . All vertex objects are stored in an unordered list V , and all edge objects are stored in an unordered list E . We illustrate an example of the edge list structure for a graph G in Figure 14.4.

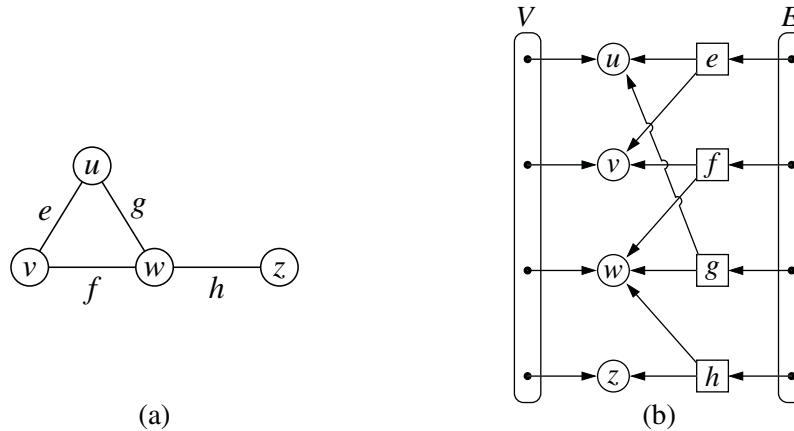


Figure 14.4: (a) A graph G ; (b) schematic representation of the edge list structure for G . Notice that an edge object refers to the two vertex objects that correspond to its endpoints, but that vertices do not refer to incident edges.

To support the many methods of the Graph ADT (Section 14.1), we assume the following additional features of an edge list representation. Collections V and E are represented with doubly linked lists using our `LinkedPositionalList` class from Chapter 7.

Vertex Objects

The vertex object for a vertex v storing element x has instance variables for:

- A reference to element x , to support the `getElement()` method.
- A reference to the position of the vertex instance in the list V , thereby allowing v to be efficiently removed from V if it were removed from the graph.

Edge Objects

The edge object for an edge e storing element x has instance variables for:

- A reference to element x , to support the `getElement()` method.
- References to the vertex objects associated with the endpoint vertices of e . These will allow for constant-time support for methods `endVertices( $e$ )` and `opposite( $v$ ,  $e$ )`.
- A reference to the position of the edge instance in list E , thereby allowing e to be efficiently removed from E if it were removed from the graph.

Performance of the Edge List Structure

The performance of an edge list structure in fulfilling the graph ADT is summarized in Table 14.2. We begin by discussing the space usage, which is $O(n + m)$ for representing a graph with n vertices and m edges. Each individual vertex or edge instance uses $O(1)$ space, and the additional lists V and E use space proportional to their number of entries.

In terms of running time, the edge list structure does as well as one could hope in terms of reporting the number of vertices or edges, or in producing an iteration of those vertices or edges. By querying the respective list V or E , the `numVertices` and `numEdges` methods run in $O(1)$ time, and by iterating through the appropriate list, the methods `vertices` and `edges` run respectively in $O(n)$ and $O(m)$ time.

The most significant limitations of an edge list structure, especially when compared to the other graph representations, are the $O(m)$ running times of methods `getEdge( $u, v$ )`, `outDegree( $v$ )`, and `outgoingEdges( $v$ )` (and corresponding methods `inDegree` and `incomingEdges`). The problem is that with all edges of the graph in an unordered list E , the only way to answer those queries is through an exhaustive inspection of all edges.

Finally, we consider the methods that update the graph. It is easy to add a new vertex or a new edge to the graph in $O(1)$ time. For example, a new edge can be added to the graph by creating an `Edge` instance storing the given element as data, adding that instance to the positional list E , and recording its resulting `Position` within E as an attribute of the edge. That stored position can later be used to locate and remove this edge from E in $O(1)$ time, and thus implement the method `removeEdge( $e$ )`.

It is worth discussing why the `removeVertex( $v$ )` method has a running time of $O(m)$. As stated in the graph ADT, when a vertex v is removed from the graph, all edges incident to v must also be removed (otherwise, we would have a contradiction of edges that refer to vertices that are not part of the graph). To locate the incident edges to the vertex, we must examine all edges of E .

Method	Running Time
<code>numVertices()</code> , <code>numEdges()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>getEdge(u, v)</code> , <code>outDegree(v)</code> , <code>outgoingEdges(v)</code>	$O(m)$
<code>insertVertex(x)</code> , <code>insertEdge(u, v, x)</code> , <code>removeEdge(e)</code>	$O(1)$
<code>removeVertex(v)</code>	$O(m)$

Table 14.2: Running times of the methods of a graph implemented with the edge list structure. The space used is $O(n + m)$, where n is the number of vertices and m is the number of edges.

14.2.2 Adjacency List Structure

The adjacency list structure for a graph adds extra information to the edge list structure that supports direct access to the incident edges (and thus to the adjacent vertices) of each vertex. Specifically, for each vertex v , we maintain a collection $I(v)$, called the **incidence collection** of v , whose entries are edges incident to v . In the case of a directed graph, outgoing and incoming edges can be respectively stored in two separate collections, $I_{\text{out}}(v)$ and $I_{\text{in}}(v)$. Traditionally, the incidence collection $I(v)$ for a vertex v is a list, which is why we call this way of representing a graph the **adjacency list** structure.

We require that the primary structure for an adjacency list maintain the collection V of vertices in a way so that we can locate the secondary structure $I(v)$ for a given vertex v in $O(1)$ time. This could be done by using a positional list to represent V , with each Vertex instance maintaining a direct reference to its $I(v)$ incidence collection; we illustrate such an adjacency list structure of a graph in Figure 14.5. If vertices can be uniquely numbered from 0 to $n - 1$, we could instead use a primary array-based structure to access the appropriate secondary lists.

The primary benefit of an adjacency list is that the collection $I(v)$ (or more specifically, $I_{\text{out}}(v)$) contains exactly those edges that should be reported by the method `outgoingEdges(v)`. Therefore, we can implement this method by iterating the edges of $I(v)$ in $O(\deg(v))$ time, where $\deg(v)$ is the degree of vertex v . This is the best possible outcome for any graph representation, because there are $\deg(v)$ edges to be reported.

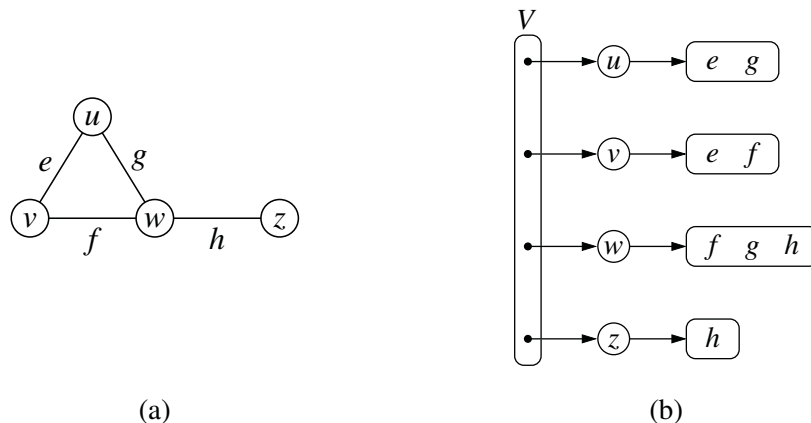


Figure 14.5: (a) An undirected graph G ; (b) a schematic representation of the adjacency list structure for G . Collection V is the primary list of vertices, and each vertex has an associated list of incident edges. Although not diagrammed as such, we presume that each edge of the graph is represented with a unique Edge instance that maintains references to its endpoint vertices, and that E is a list of all edges.

Performance of the Adjacency List Structure

Table 14.3 summarizes the performance of the adjacency list structure implementation of a graph, assuming that the primary collection V and E , and all secondary collections $I(v)$ are implemented with doubly linked lists.

Asymptotically, the space requirements for an adjacency list are the same as an edge list structure, using $O(n + m)$ space for a graph with n vertices and m edges. It is clear that the primary lists of vertices and edges use $O(n + m)$ space. In addition, the sum of the lengths of all secondary lists is $O(m)$, for reasons that were formalized in Propositions 14.8 and 14.9. In short, an undirected edge (u, v) is referenced in both $I(u)$ and $I(v)$, but its presence in the graph results in only a constant amount of additional space.

We have already noted that the `outgoingEdges( $v$ )` method can be achieved in $O(\deg(v))$ time based on use of $I(v)$. For a directed graph, this is more specifically $O(\text{outdeg}(v))$ based on use of $I_{\text{out}}(v)$. The `outDegree( $v$ )` method of the graph ADT can run in $O(1)$ time, assuming collection $I(v)$ can report its size in similar time. To locate a specific edge for implementing `getEdge( $u, v$ )`, we can search through either $I(u)$ and $I(v)$ (or for a directed graph, either $I_{\text{out}}(u)$ or $I_{\text{in}}(v)$). By choosing the smaller of the two, we get $O(\min(\deg(u), \deg(v)))$ running time.

The rest of the bounds in Table 14.3 can be achieved with additional care. To efficiently support deletions of edges, an edge (u, v) would need to maintain a reference to its positions within both $I(u)$ and $I(v)$, so that it could be deleted from those collections in $O(1)$ time. To remove a vertex v , we must also remove any incident edges, but at least we can locate those edges in $O(\deg(v))$ time.

Method	Running Time
<code>numVertices()</code> , <code>numEdges()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>getEdge(u, v)</code>	$O(\min(\deg(u), \deg(v)))$
<code>outDegree(v)</code> , <code>inDegree(v)</code>	$O(1)$
<code>outgoingEdges(v)</code> , <code>incomingEdges(v)</code>	$O(\deg(v))$
<code>insertVertex(x)</code> , <code>insertEdge(u, v, x)</code>	$O(1)$
<code>removeEdge(e)</code>	$O(1)$
<code>removeVertex(v)</code>	$O(\deg(v))$

Table 14.3: Running times of the methods of a graph implemented with the adjacency list structure. The space used is $O(n + m)$, where n is the number of vertices and m is the number of edges.

14.2.3 Adjacency Map Structure

In the adjacency list structure, we assume that the secondary incidence collections are implemented as unordered linked lists. Such a collection $I(v)$ uses space proportional to $O(\deg(v))$, allows an edge to be added or removed in $O(1)$ time, and allows an iteration of all edges incident to vertex v in $O(\deg(v))$ time. However, the best implementation of $\text{getEdge}(u, v)$ requires $O(\min(\deg(u), \deg(v)))$ time, because we must search through either $I(u)$ or $I(v)$.

We can improve the performance by using a hash-based map to implement $I(v)$ for each vertex v . Specifically, we let the opposite endpoint of each incident edge serve as a key in the map, with the edge structure serving as the value. We call such a graph representation an **adjacency map**. (See Figure 14.6.) The space usage for an adjacency map remains $O(n + m)$, because $I(v)$ uses $O(\deg(v))$ space for each vertex v , as with the adjacency list.

The advantage of the adjacency map, relative to an adjacency list, is that the $\text{getEdge}(u, v)$ method can be implemented in **expected** $O(1)$ time by searching for vertex u as a key in $I(v)$, or vice versa. This provides a likely improvement over the adjacency list, while retaining the worst-case bound of $O(\min(\deg(u), \deg(v)))$.

In comparing the performance of adjacency map to other representations (see Table 14.1), we find that it essentially achieves optimal running times for all methods, making it an excellent all-purpose choice as a graph representation.

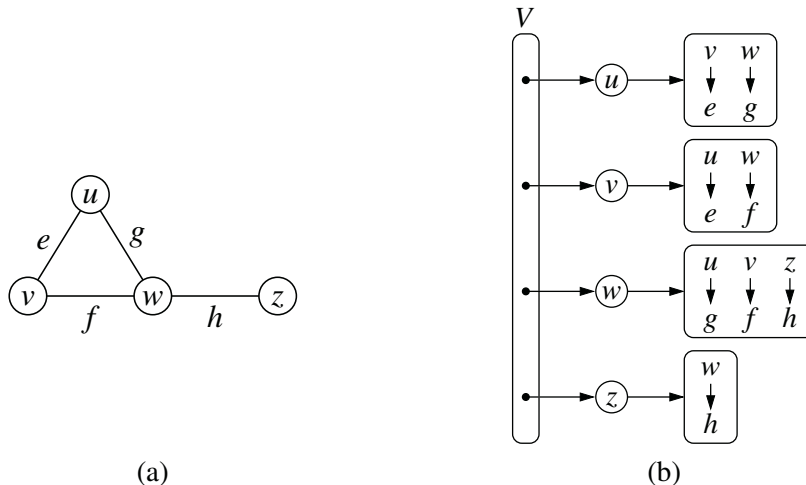


Figure 14.6: (a) An undirected graph G ; (b) a schematic representation of the adjacency map structure for G . Each vertex maintains a secondary map in which neighboring vertices serve as keys, with the connecting edges as associated values. As with the adjacency list, we presume that there is also an overall list E of all Edge instances.

14.2.4 Adjacency Matrix Structure

The **adjacency matrix** structure for a graph G augments the edge list structure with a matrix A (that is, a two-dimensional array, as in Section 3.1.5), which allows us to locate an edge between a given pair of vertices in *worst-case* constant time. In the adjacency matrix representation, we think of the vertices as being the integers in the set $\{0, 1, \dots, n-1\}$ and the edges as being pairs of such integers. This allows us to store references to edges in the cells of a two-dimensional $n \times n$ array A . Specifically, the cell $A[i][j]$ holds a reference to the edge (u, v) , if it exists, where u is the vertex with index i and v is the vertex with index j . If there is no such edge, then $A[i][j] = \text{null}$. We note that array A is symmetric if graph G is undirected, as $A[i][j] = A[j][i]$ for all pairs i and j . (See Figure 14.7.)

The most significant advantage of an adjacency matrix is that any edge (u, v) can be accessed in worst-case $O(1)$ time; recall that the adjacency map supports that operation in $O(1)$ *expected* time. However, several operations are less efficient with an adjacency matrix. For example, to find the edges incident to vertex v , we must presumably examine all n entries in the row associated with v ; recall that an adjacency list or map can locate those edges in optimal $O(\deg(v))$ time. Adding or removing vertices from a graph is problematic, as the matrix must be resized.

Furthermore, the $O(n^2)$ space usage of an adjacency matrix is typically far worse than the $O(n + m)$ space required of the other representations. Although, in the worst case, the number of edges in a **dense** graph will be proportional to n^2 , most real-world graphs are **sparse**. In such cases, use of an adjacency matrix is inefficient. However, if a graph is dense, the constants of proportionality of an adjacency matrix can be smaller than that of an adjacency list or map. In fact, if edges do not have auxiliary data, a boolean adjacency matrix can use one bit per edge slot, such that $A[i][j] = \text{true}$ if and only if associated (u, v) is an edge.

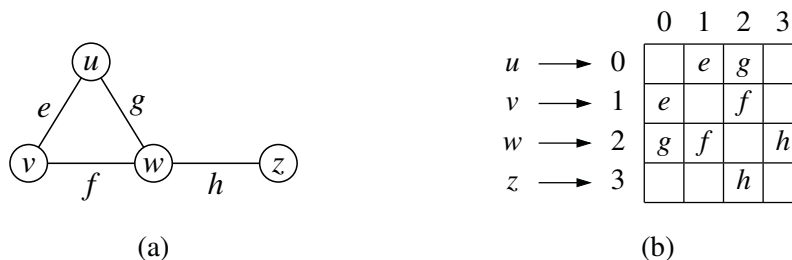


Figure 14.7: (a) An undirected graph G ; (b) a schematic representation of the auxiliary adjacency matrix structure for G , in which n vertices are mapped to indices 0 to $n-1$. Although not diagrammed as such, we presume that there is a unique Edge instance for each edge, and that it maintains references to its endpoint vertices. We also assume that there is a secondary edge list (not pictured), to allow the `edges()` method to run in $O(m)$ time, for a graph with m edges.

14.2.5 Java Implementation

In this section, we provide an implementation of the Graph ADT, based on the *adjacency map* representation, as described in Section 14.2.3. We use positional lists to represent each of the primary lists V and E , as originally described in the edge list representation. Additionally, for each vertex v , we use a hash-based map to represent the secondary incidence map $I(v)$.

To gracefully support both undirected and directed graphs, each vertex maintains two different map references: outgoing and incoming. In the directed case, these are initialized to two distinct map instances, representing $I_{\text{out}}(v)$ and $I_{\text{in}}(v)$, respectively. In the case of an undirected graph, we assign both outgoing and incoming as aliases to a single map instance.

Our implementation is organized as follows. We assume definitions for `Vertex`, `Edge`, and `Graph` interfaces, although for the sake of brevity, we do not include those definitions in the book (they are available online). We then define a concrete `AdjacencyMapGraph` class, with nested classes `InnerVertex` and `InnerEdge` to implement the vertex and edge abstractions. These classes use generic parameters V and E to designate the element type stored respectively at vertices and edges.

We begin in Code Fragment 14.1, with the definitions of the `InnerVertex` and `InnerEdge` classes (although in reality, those definitions should be nested within the following `AdjacencyMapGraph` class). Note well how the `InnerVertex` constructor initializes the outgoing and incoming instance variables depending on whether the overall graph is undirected or directed.

Code Fragments 14.2 and 14.3 contain the core implementation of the class `AdjacencyMapGraph`. A graph instance maintains a boolean variable that designates whether the graph is directed, and it maintains the vertex list and edge list. Although not shown in these code fragments, our implementation includes private `validate` methods that perform type conversions between the public `Vertex` and `Edge` interface types to the concrete `InnerVertex` and `InnerEdge` classes, while also performing some error checking. This design is similar to the `validate` method of the `LinkedPositionalList` class (see Code Fragment 7.10 of Section 7.3.3), which converts an outward `Position` to the underlying `Node` type for that class.

The most complex methods are those that modify the graph. When `insertVertex` is called, we must create a new `InnerVertex` instance, add that vertex to the list of vertices, and record its position within that list (so that we can efficiently delete it from the list if the vertex is removed from the graph). When inserting an edge (u, v) , we must also create a new instance, add it to the edge list, and record its position, yet we must also add the new edge to the outgoing adjacency map for vertex u , and the incoming map for vertex v . Code Fragment 14.3 contains code for `removeVertex` as well; the implementation of `removeEdge` is not included, but is available in the online version of the code.


```

1  /** A vertex of an adjacency map graph representation. */
2  private class InnerVertex<V> implements Vertex<V> {
3      private V element;
4      private Position<Vertex<V>> pos;
5      private Map<Vertex<V>, Edge<E>> outgoing, incoming;
6      /** Constructs a new InnerVertex instance storing the given element. */
7      public InnerVertex(V elem, boolean graphsDirected) {
8          element = elem;
9          outgoing = new ProbeHashMap<>();
10         if (graphsDirected)
11             incoming = new ProbeHashMap<>();
12         else
13             incoming = outgoing;          // if undirected, alias outgoing map
14     }
15     /** Returns the element associated with the vertex. */
16     public V getElement() { return element; }
17     /** Stores the position of this vertex within the graph's vertex list. */
18     public void setPosition(Position<Vertex<V>> p) { pos = p; }
19     /** Returns the position of this vertex within the graph's vertex list. */
20     public Position<Vertex<V>> getPosition() { return pos; }
21     /** Returns reference to the underlying map of outgoing edges. */
22     public Map<Vertex<V>, Edge<E>> getOutgoing() { return outgoing; }
23     /** Returns reference to the underlying map of incoming edges. */
24     public Map<Vertex<V>, Edge<E>> getIncoming() { return incoming; }
25 } //----- end of InnerVertex class -----
26
27 /** An edge between two vertices. */
28 private class InnerEdge<E> implements Edge<E> {
29     private E element;
30     private Position<Edge<E>> pos;
31     private Vertex<V>[] endpoints;
32     /** Constructs InnerEdge instance from u to v, storing the given element. */
33     public InnerEdge(Vertex<V> u, Vertex<V> v, E elem) {
34         element = elem;
35         endpoints = (Vertex<V>[]) new Vertex[ ]{u,v}; // array of length 2
36     }
37     /** Returns the element associated with the edge. */
38     public E getElement() { return element; }
39     /** Returns reference to the endpoint array. */
40     public Vertex<V>[] getEndpoints() { return endpoints; }
41     /** Stores the position of this edge within the graph's vertex list. */
42     public void setPosition(Position<Edge<E>> p) { pos = p; }
43     /** Returns the position of this edge within the graph's vertex list. */
44     public Position<Edge<E>> getPosition() { return pos; }
45 } //----- end of InnerEdge class -----

```

Code Fragment 14.1: InnerVertex and InnerEdge classes (to be nested within the AdjacencyMapGraph class). Interfaces Vertex<V> and Edge<E> are not shown.

```

1  public class AdjacencyMapGraph<V,E> implements Graph<V,E> {
2      // nested InnerVertex and InnerEdge classes defined here...
3      private boolean isDirected;
4      private PositionalList<Vertex<V>> vertices = new LinkedPositionalList<>();
5      private PositionalList<Edge<E>> edges = new LinkedPositionalList<>();
6      /** Constructs an empty graph (either undirected or directed). */
7      public AdjacencyMapGraph(boolean directed) { isDirected = directed; }
8      /** Returns the number of vertices of the graph */
9      public int numVertices() { return vertices.size(); }
10     /** Returns the vertices of the graph as an iterable collection */
11     public Iterable<Vertex<V>> vertices() { return vertices; }
12     /** Returns the number of edges of the graph */
13     public int numEdges() { return edges.size(); }
14     /** Returns the edges of the graph as an iterable collection */
15     public Iterable<Edge<E>> edges() { return edges; }
16     /** Returns the number of edges for which vertex v is the origin. */
17     public int outDegree(Vertex<V> v) {
18         InnerVertex<V> vert = validate(v);
19         return vert.getOutgoing().size();
20     }
21     /** Returns an iterable collection of edges for which vertex v is the origin. */
22     public Iterable<Edge<E>> outgoingEdges(Vertex<V> v) {
23         InnerVertex<V> vert = validate(v);
24         return vert.getOutgoing().values(); // edges are the values in the adjacency map
25     }
26     /** Returns the number of edges for which vertex v is the destination. */
27     public int inDegree(Vertex<V> v) {
28         InnerVertex<V> vert = validate(v);
29         return vert.getIncoming().size();
30     }
31     /** Returns an iterable collection of edges for which vertex v is the destination. */
32     public Iterable<Edge<E>> incomingEdges(Vertex<V> v) {
33         InnerVertex<V> vert = validate(v);
34         return vert.getIncoming().values(); // edges are the values in the adjacency map
35     }
36     public Edge<E> getEdge(Vertex<V> u, Vertex<V> v) {
37         /** Returns the edge from u to v, or null if they are not adjacent. */
38         InnerVertex<V> origin = validate(u);
39         return origin.getOutgoing().get(v); // will be null if no edge from u to v
40     }
41     /** Returns the vertices of edge e as an array of length two. */
42     public Vertex<V>[] endVertices(Edge<E> e) {
43         InnerEdge<E> edge = validate(e);
44         return edge.getEndpoints();
45     }

```

Code Fragment 14.2: AdjacencyMapGraph class definition. (Continues in Code Fragment 14.3.) The validate(v) and validate(e) methods are available online.

```

46  /** Returns the vertex that is opposite vertex v on edge e. */
47  public Vertex<V> opposite(Vertex<V> v, Edge<E> e)
48                                     throws IllegalArgumentException {
49      InnerEdge<E> edge = validate(e);
50      Vertex<V>[] endpoints = edge.getEndpoints();
51      if (endpoints[0] == v)
52          return endpoints[1];
53      else if (endpoints[1] == v)
54          return endpoints[0];
55      else
56          throw new IllegalArgumentException("v is not incident to this edge");
57  }
58  /** Inserts and returns a new vertex with the given element. */
59  public Vertex<V> insertVertex(V element) {
60      InnerVertex<V> v = new InnerVertex<>(element, isDirected);
61      v.setPosition(vertices.addLast(v));
62      return v;
63  }
64  /** Inserts and returns a new edge between u and v, storing given element. */
65  public Edge<E> insertEdge(Vertex<V> u, Vertex<V> v, E element)
66                                     throws IllegalArgumentException {
67      if (getEdge(u,v) == null) {
68          InnerEdge<E> e = new InnerEdge<>(u, v, element);
69          e.setPosition(edges.addLast(e));
70          InnerVertex<V> origin = validate(u);
71          InnerVertex<V> dest = validate(v);
72          origin.getOutgoing().put(v, e);
73          dest.getIncoming().put(u, e);
74          return e;
75      } else
76          throw new IllegalArgumentException("Edge from u to v exists");
77  }
78  /** Removes a vertex and all its incident edges from the graph. */
79  public void removeVertex(Vertex<V> v) {
80      InnerVertex<V> vert = validate(v);
81      // remove all incident edges from the graph
82      for (Edge<E> e : vert.getOutgoing().values())
83          removeEdge(e);
84      for (Edge<E> e : vert.getIncoming().values())
85          removeEdge(e);
86      // remove this vertex from the list of vertices
87      vertices.remove(vert.getPosition());
88  }
89  }

```

Code Fragment 14.3: AdjacencyMapGraph class definition (continued from Code Fragment 14.2). We omit the removeEdge method, for brevity.